2027考研重邮802数据结构模拟试卷
第2套 参考答案

对应第6版题面。程序填空接受等价写法；非唯一的树形、编码或路径按题内规则判分。算法实现仅给出一种正确方法，其他满足题目约束的解法同样有效。

评分建议用于核对作答要点。复杂度须与考生实际实现一致；不能仅因实现与参考代码不同而扣分。

一、程序阅读题

1. （5分）

__1__ = 7；__2__ = 2M(n-1)+1；__3__ = 2^n。

两次规模为 n-1 的递归和一次 move 给出递推式。由 M(1)=1 得 M(n)=2^n-1，因此 n=3 时执行 7 次。每次移动本身为 O(1)，总时间 O(2^n)；递归栈深度为 n。

2. （5分）

__1__ = q；__2__ = y；__3__ = q。

交换后局部链为 x<->p<->q<->y：p->next 指向 q，q->next 指向 y，且 y 的前驱变为 q；其余三条指针由原代码更新。示例前向输出从 10 20 变为 20 10，tail.prior 为 10。

3. （5分）

__1__ = s->topL + 1 == s->topR；__2__ = --s->topR；__3__ = s->topR++。

左栈顶右边紧邻右栈顶即满。右栈入栈先向左移动 topR，再写数组；右栈出栈先取原 topR 处元素，再向右移动。示例出栈值为 8，最后 topL=0, topR=5。

4. （5分）

__1__ = 0；__2__ 可填 a - b > 1 || b - a > 1；__3__ = 1 + (a > b ? a : b)。

空树高度为 0；任一子树返回 -1 表示不平衡，随后检查左右高度差是否超过 1。若平衡，当前高度为较大子树高度加 1。示例输出 1 0 1。

二、填空题

1. （3分）

__1__ = 14。依次算 8-3=5、5×2=10、10+4=14；弹栈时先取的是右操作数。

2. （3分）

__1__ = 14。四个不同元素依次入栈的合法出栈序列数是第 4 个 Catalan 数：C_4 = (1/5)·C(8,4)=14。

3. （3分）

__1__ = 3；__2__ = 2。队长 (rear-front+8) mod 8=(1-6+8) mod 8=3；入队后 rear=(1+1) mod 8=2。

4. （3分）

__1__ = 5；__2__ = 0。P[1..5]=aaaaa 的最长真前后缀为 aaaa，故 next[6]=4+1=5；整个 aaaaab 的末字符为 b，非空前缀均以 a 开始，故 π(6)=0。

5. （3分）

__1__ = 468。从首元素到 A[2,3,2] 的元素偏移量为 (2-1)·4·3+(3-2)·3+(2-0)=17，字节偏移量为 17×4=68，地址为 400+68。

6. （3分）

__1__ = 3。森林中每棵树都有“边数=结点数-1”，总边数为 12-树数=9，故有 3 棵树。

7. （3分）

__1__ = n+1。共有 2n 个孩子指针；无共享结点的二叉树有 n-1 条孩子边，因此空指针数为 2n-(n-1)=n+1。

8. （3分）

__1__ = 5。以题定下取整中点划分，16 个关键字的成功判定树最深结点比较 5 次；例如最右端关键字依次经过下标 7、11、13、14、15。

9. （3分）

__1__ = 3；__2__ = 2。12 先放下标7，与下标3的15比较并交换；再与下标1的10比较后停止，所以最终在下标3，共比较2次。

10. （3分）

__1__ = 8。严格二叉树设叶数 L、双孩子结点数 I，则 L=I+1，L+I=15，解得 L=8。

三、综合应用题

1．（10分）

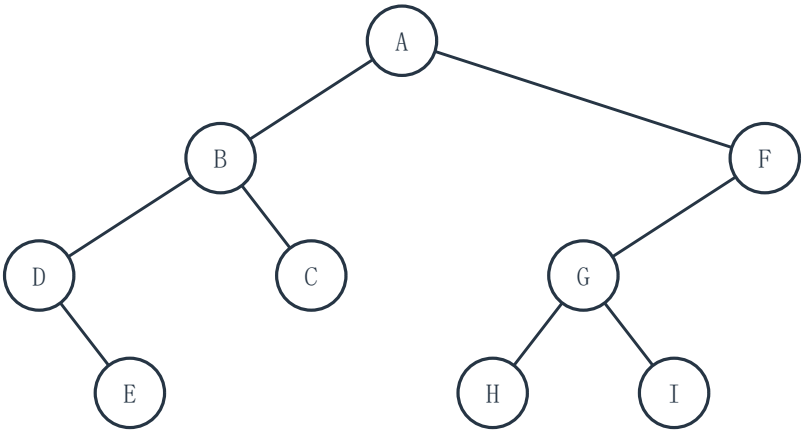
（1）按左孩子、右孩子次序列出：

结点	A	B	C	D	E	F	G	H	I
左孩子	B	D	NULL	NULL	NULL	G	H	NULL	NULL
右孩子	F	C	NULL	E	NULL	NULL	I	NULL	NULL

（2）原森林先根遍历为 **A B D E C F G H I**；后根遍历为 **D E B C A H G I F**。

（3）根为 A、F，共 **2** 棵树；边数为 **7**。

先根遍历每棵树时先访问根再自左至右递归孩子；后根遍历先访问孩子再访问根。转换中的 A→F 是根之间的右兄弟关系，不是森林的一条父子边。



转换后的孩子兄弟二叉树

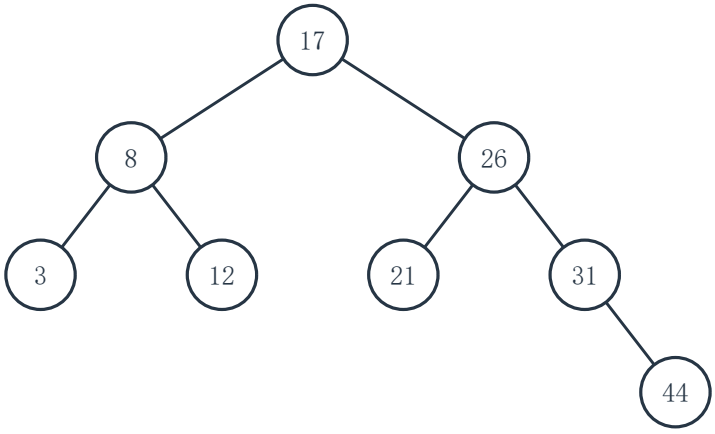
2. （10分）

（1）成功判定树（括号内为左右孩子）如下：

17:(8,26)
 8:(3,12)　　26:(21,31)
 3:(NULL,NULL)　　12:(NULL,NULL)
21:(NULL,NULL)　　31:(NULL,44)
44:(NULL,NULL)

（2）按比较次数分组：第 1 层 1 个，第 2 层 2 个，第 3 层 4 个，第 4 层 1 个。成功 ASL=(1×1+2×2+3×4+4×1)/8=21/8。

（3）九个失败间隙自左至右的比较次数为 3，3，3，3，3，3，3，4，4。失败 ASL=(7×3+2×4)/9=29/9。



折半查找判定树

3．（10分）

（1）线性探测表为：

槽号	0	1	2	3	4	5	6	7	8	9	10
关键字	空	12	23	34	4	15	26	7	空	空	空

拉链法的非空桶：1:12→23→34，4:4→15→26，7:7。

（2）线性探测按题目给定七个关键字的成功比较次数为 1,2,3,1,2,3,1，成功 ASL=13/7。拉链法对应链内位置也为 1,2,3,1,2,3,1，成功 ASL=13/7。

（3）线性探测按起始地址 0～10 的失败比较次数为 1,8,7,6,5,4,3,2,1,1,1，失败 ASL=39/11。拉链法按桶号 0～10 的失败比较次数为 0,3,0,0,3,0,0,1,0,0,0，失败 ASL=7/11。线性探测把首个空槽计入，拉链法空桶不计比较。

4．（10分）

（1）逐轮选入顶点并松弛后的暂定距离如下：

选入	B	C	D	E	F
A	0	4	∞	∞	∞
B	0	2	7	∞	∞
C	0	2	3	8	∞
D	0	2	3	6	∞
E	0	2	3	6	∞

因此选入顺序为 **A，B，C，D，E**；F 不可达，不属于题目要求列出的轮次。

（2）最终距离：d(A)=0，d(B)=0，d(C)=2，d(D)=3，d(E)=6，d(F)=∞。

（3）一条 A 到 E 的最短路径为 **A→B→C→D→E**，长度 0+2+1+3=6。

5．（10分）

（1）按事件 0～5 顺序，最早发生时间 ve=(0,3,2,6,6,8)。例如事件3取 max(3+2,2+4)=6，事件5取 max(3+3,6+2,6+1)=8。

（2）最迟发生时间 vl=(0,4,2,6,7,8)。从汇点5的8逆推：vl(3)=8-2=6，vl(4)=8-1=7，vl(1)=min(6-2,7-3,8-3)=4。

（3）工期 8。活动 B（0→2）、E（2→3）、G（3→5）的最早开始时间等于最迟开始时间，故关键活动为 B、E、G；全部关键路径仅 0→2→3→5。

6. （10分）

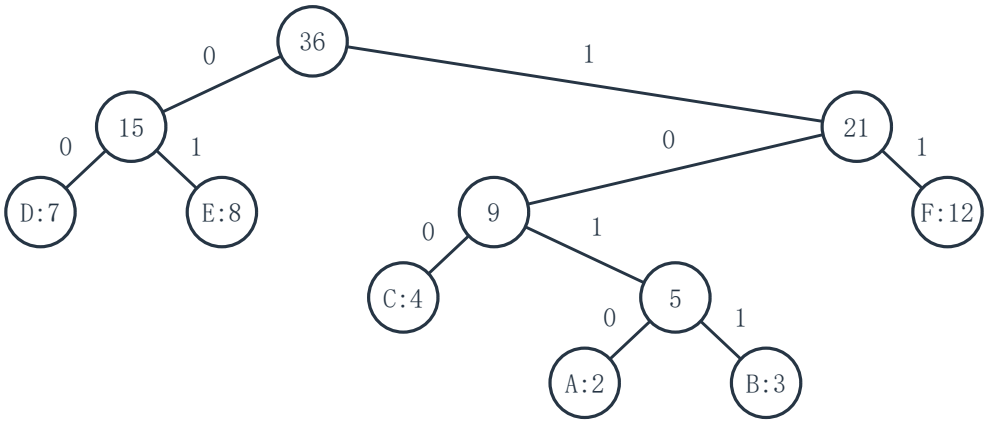
（1）合并顺序：2+3=5，4+5=9，7+8=15，9+12=21，15+21=36。

（2）一组符合“小权值左0、大权值右1”的码字为：

符号	A	B	C	D	E	F
码字	1010	1011	100	00	01	11

对应树的根左右子树权值分别为15（由 D7、E8 构成）和21（由9、F12构成）；权值9又由 C4 与 (A2,B3) 权值5 构成。

（3）WPL=2×4+3×4+4×3+7×2+8×2+12×2=86，也等于五次合并权值之和 5+9+15+21+36=86。



一组Huffman树（叶结点为字符:频率，左0右1）

7．（10分）

（1）甲完成 i=4 后：[17,24,36,53,72,91,45,68]。

（2）甲完成 i=6 后：[17,24,36,45,53,72,91,68]。i=5 处理91不移动，i=6 将45插到36与53之间。

（3）乙第一趟挖坑划分后：[45,17,36,24,53,91,72,68]。

（4）枢轴53最后处于下标 4。从右侧先搬45到左端，再从左侧搬72到右端；继续从右侧搬36到左侧，两个指针在下标4相遇，填入53。

四、算法设计与分析题

1．（8分）

基本思想。遍历一次原链，取下当前数据结点，按数据是否小于0依次接到“负数链”或“非负数链”的尾部；两条链内部始终按原扫描顺序追加。最后把负数链接到非负数链前，并从原头结点接入。两个临时哨兵是局部变量，不新建数据结点。

```
#include <stddef.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;

void stableNegFirst(Node *L) {
    Node neg = {0, NULL}, nonneg = {0, NULL};
    Node *tn = &neg, *tp = &nonneg;
    Node *p = L->next;
    while (p != NULL) {
        Node *next = p->next;
        p->next = NULL;
        if (p->data < 0) {
            tn->next = p;
            tn = p;
        } else {
            tp->next = p;
            tp = p;
        }
        p = next;
    }
    tn->next = nonneg.next;
    L->next = neg.next;
}
```

复杂度。n 为数据结点数，每个结点只访问一次，时间 O(n)，辅助空间 O(1)。空表或无负数时 tn 仍指向局部哨兵，连接后 L->next 正确；无非负数时末尾接 NULL。

评分建议（8分）：稳定分组思想1分；正确拆链、尾插和最终连接5分；时间复杂度1分；辅助空间复杂度1分。未保持两组各自原序的实现不能取得稳定连接相关分。

2. （10分）

基本思想。后序遍历，每个结点先取得左右子树的叶数；若左右孩子都为空，写入1，否则写入左右结果之和。递归返回值就是当前子树叶数，因此每个结点只处理一次，孩子指针保持原值。

```
#include <stddef.h>
typedef struct TNode {
    int data;
    int count;
    struct TNode *left, *right;
} TNode;

static int leafCount(TNode *p) {
    if (p == NULL) return 0;
    if (p->left == NULL && p->right == NULL) {
        p->count = 1;
        return 1;
    }
    int a = leafCount(p->left);
    int b = leafCount(p->right);
    p->count = a + b;
    return p->count;
}

void fillLeafCounts(TNode *root) {
    (void)leafCount(root);
}
```

复杂度。n 为结点数、h 为树高，时间 O(n)，辅助空间 O(h)（递归调用栈；空树 O(1)，链状树最坏 O(n)）。

评分建议（10分）：后序汇总思想2分；完整可执行代码6分，其中空树/叶结点及内部结点写值均应正确；时间复杂度1分；计入递归栈的空间复杂度1分。

3. （12分）

基本思想。无向图中从 s 到 t 存在边数不超过 k 的路径，当且仅当最短路边数不超过 k。用 BFS 求最短路和前驱；BFS 第一次到达 t 时得到一条最短路，其 BFS 树路径天然不重复顶点。若 s=t，路径只有 s；若 k<0，按题意直接失败。逆着前驱回溯后反向写入 path。

```
#include <stddef.h>
typedef struct Arc {
    int to;
    struct Arc *next;
} Arc;
typedef struct {
    int V, E;
    Arc **head;
} Graph;

int pathAtMostK(const Graph *G, int s, int t, int k,
                int *path, int *pathLen) {
    *pathLen = 0;
    if (k < 0) return 0;
    if (s == t) {
        path[0] = s;
        *pathLen = 1;
        return 1;
    }

    int dist[100], parent[100], queue[100], rev[100];
    for (int i = 0; i < G->V; ++i) {
        dist[i] = -1;
        parent[i] = -1;
    }
    int front = 0, rear = 0;
    dist[s] = 0;
    queue[rear++] = s;
    while (front < rear && dist[t] == -1) {
        int u = queue[front++];
        for (const Arc *a = G->head[u]; a != NULL; a = a->next) {
            int v = a->to;
            if (dist[v] != -1) continue;
            dist[v] = dist[u] + 1;
            parent[v] = u;
            queue[rear++] = v;
        }
    }
    if (dist[t] == -1 || dist[t] > k) return 0;

    int len = 0;
    for (int v = t; v != -1; v = parent[v]) rev[len++] = v;
    for (int i = 0; i < len; ++i) path[i] = rev[len - 1 - i];
    *pathLen = len;
    return 1;
}
```

复杂度。BFS 至多扫描 V 个顶点及邻接表中 2E 条弧，时间 O(V+E)；dist、parent、队列与倒序路径数组合计 O(V) 辅助空间，输出数组由调用者提供。若 s=t 或 k<0，提前返回为 O(1)。

评分建议（12分）：把存在性化为 BFS 最短路并处理边界2分；初始化、访问与前驱记录5分；限长判断及无重复路径回溯3分；时间与空间复杂度各1分。仅判断可达而未输出合法路径不能取得回溯部分分。